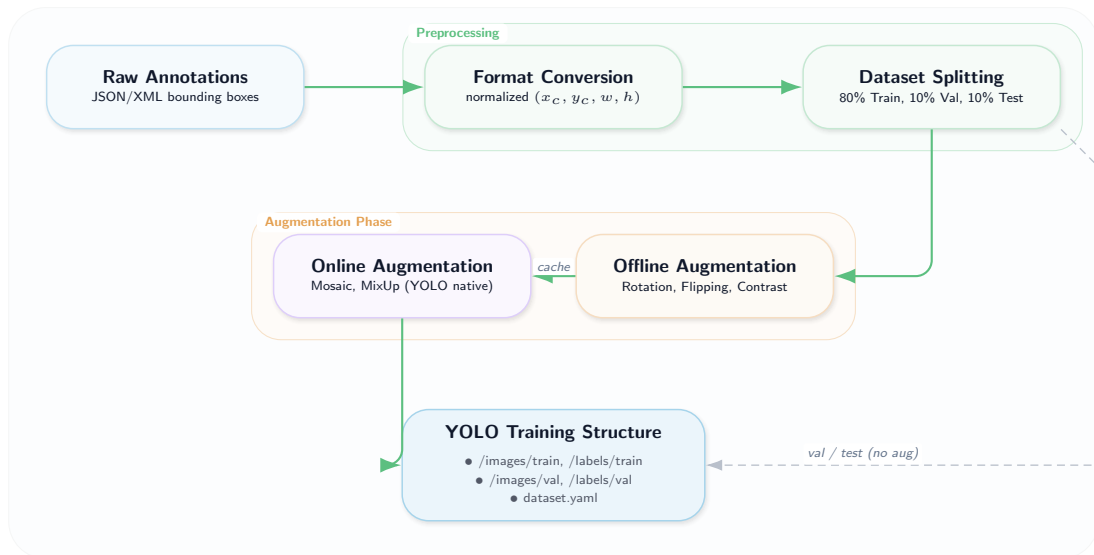


YOLO Data Pipeline Flow

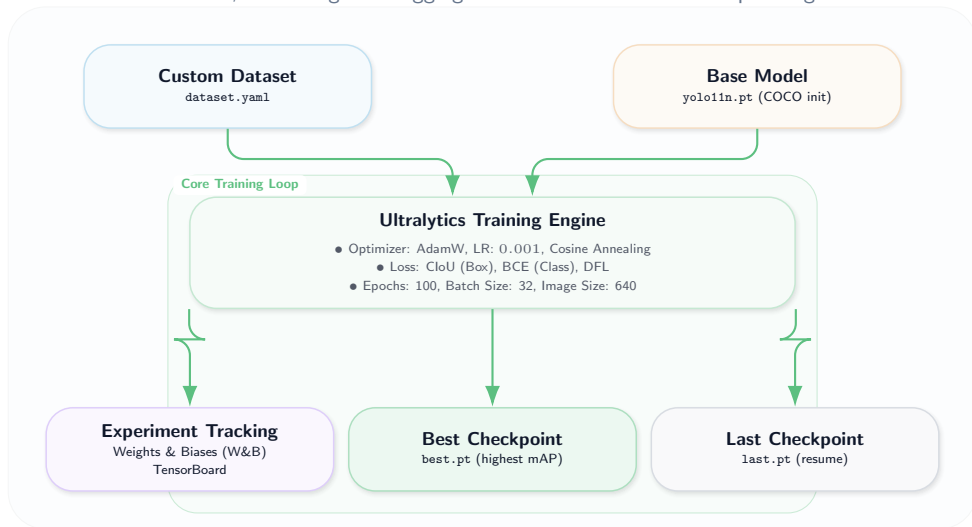
Conversion of annotations to YOLO format, train/val/test splitting, and online/offline image augmentations for robust object detection.



Augmentation Strategy — Offline augmentation expands the static dataset to combat class imbalance. Online augmentation (Mosaic, MixUp) is applied dynamically during YOLO training by the dataloader to improve model generalization and detection robustness on variable scales.

YOLO Training Pipeline Flow

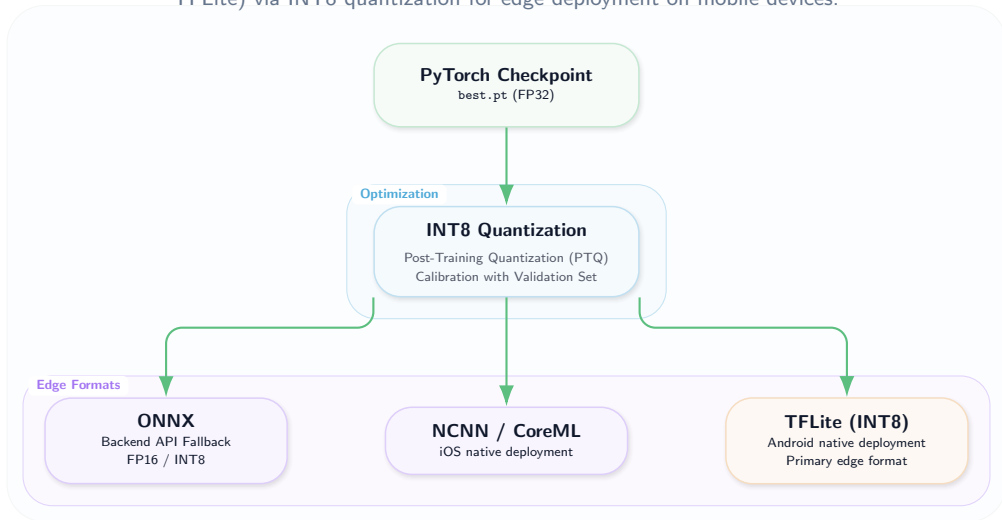
Ultralytics engine fine-tuning YOLO11n on the custom agricultural dataset, with integrated logging and automated model checkpointing.



Training Dynamics — Transfer learning from COCO pre-trained weights significantly accelerates convergence. The engine tracks mAP@0.5:0.95 on the validation set at every epoch, automatically saving best.pt to prevent overfitting.

YOLO Quantization & Export Flow

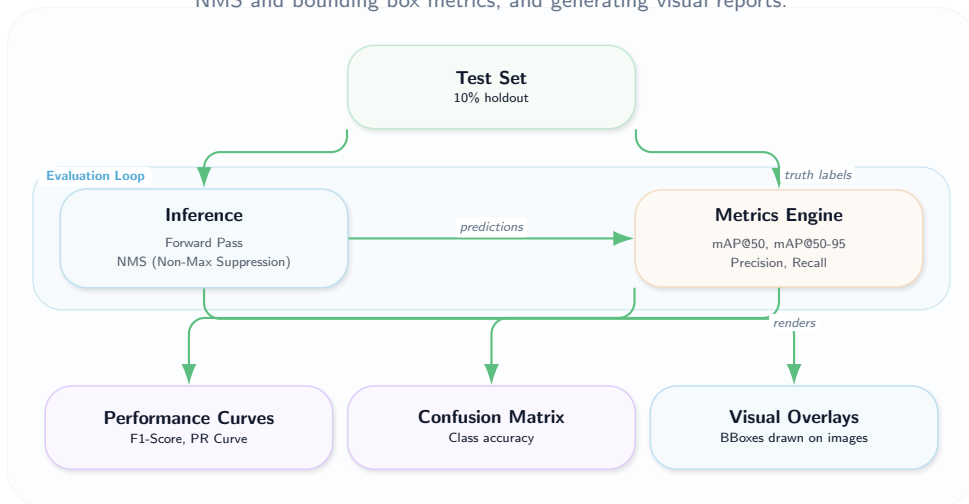
Transforming the FP32 PyTorch checkpoint into optimized formats (ONNX, TFLite) via INT8 quantization for edge deployment on mobile devices.



Edge Optimization — Quantizing from 32-bit floats to 8-bit integers drastically reduces model size (usually by 4x) and speeds up inference on mobile CPUs/NPUs. The validation set is used during calibration to compute the scaling factors, minimizing the inevitable drop in accuracy.

YOLO Validation & Evaluation Flow

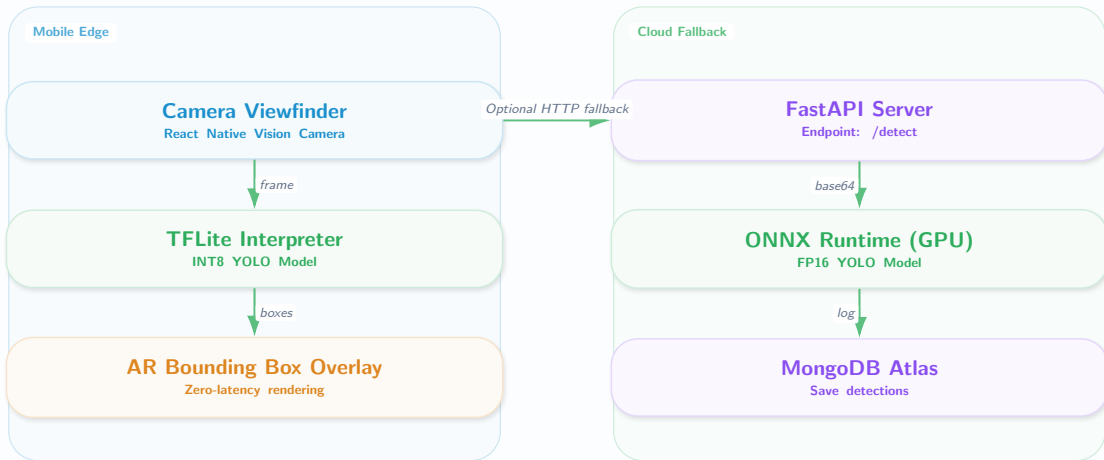
Evaluating the fine-tuned model against the test set, computing NMS and bounding box metrics, and generating visual reports.



Evaluation — Non-Maximum Suppression (NMS) filters overlapping bounding boxes before predictions are compared to ground truth. Key metrics like mAP (Mean Average Precision) at various IoU thresholds provide a holistic view of the model's localization and classification accuracy.

YOLO Deployment Architecture Flow

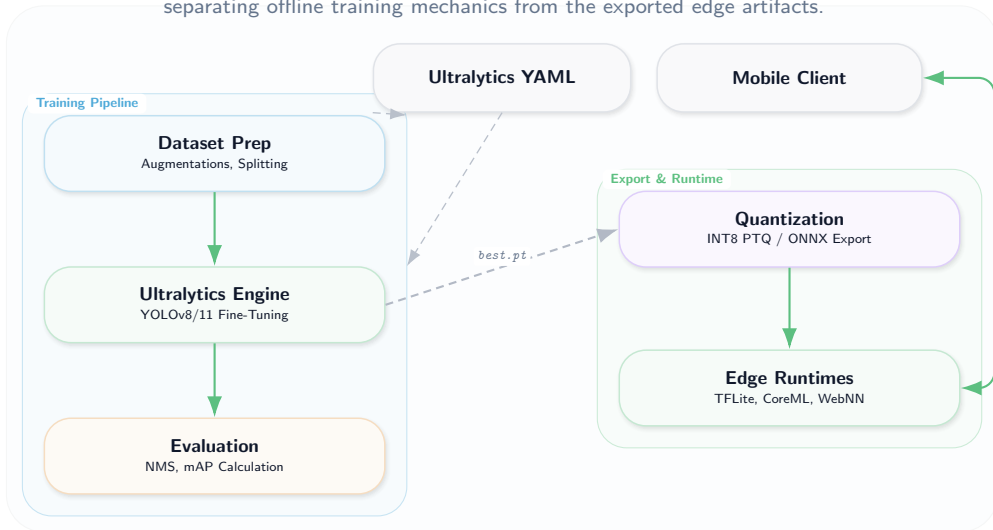
Dual deployment strategy: on-device edge inference for real-time overlays, with an optional server fallback for heavier model execution and centralized logging.



Strategy — The primary detection runs locally on the mobile device (Edge) via TFLite, providing instant visual feedback without network latency. The backend ONNX Runtime serves as an optional high-accuracy fallback or for logging diagnostic imagery when device constraints prevent local processing.

YOLO System Architecture Overview

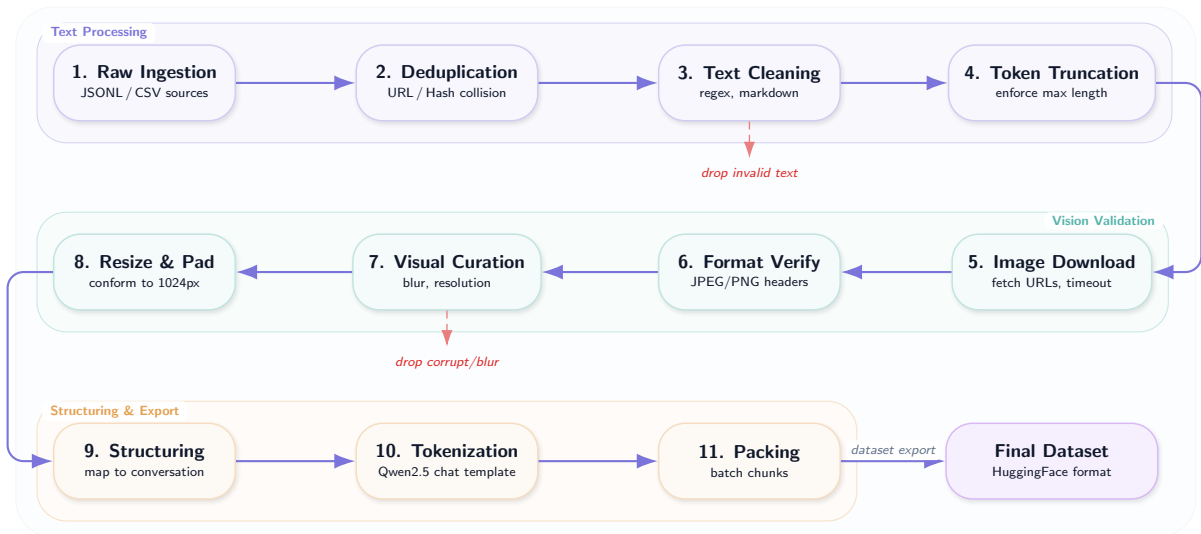
The high-level architecture of the YOLO computer vision pipeline, separating offline training mechanics from the exported edge artifacts.



Architecture — The YOLO architecture is cleanly split. The ML lifecycle handles the heavy lifting of data preparation and model training using PyTorch. The resultant artifacts are quantized and exported, decoupling the Python-based training from the lightweight C++/Java/Swift runtimes used on mobile edges.

VLM Data Pipeline Flow (11 Stages)

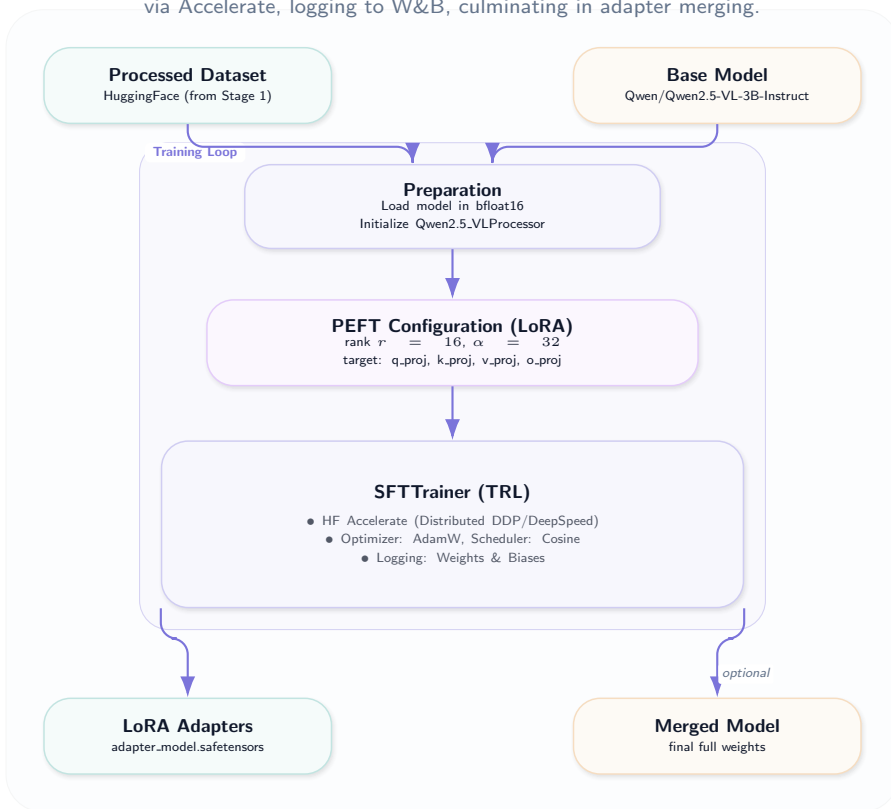
The serpentine sequence of ingestion, text cleaning, visual curation, and formatting into Qwen-style conversational arrays.



Design rationale — The pipeline is strictly sequential. Text cleaning occurs before image downloading to save bandwidth by immediately dropping malformed textual records. Visual curation guarantees that the model receives only crisp, adequately sized imagery, which is critical for the visual token representations in the Qwen architecture.

VLM Training Pipeline Flow

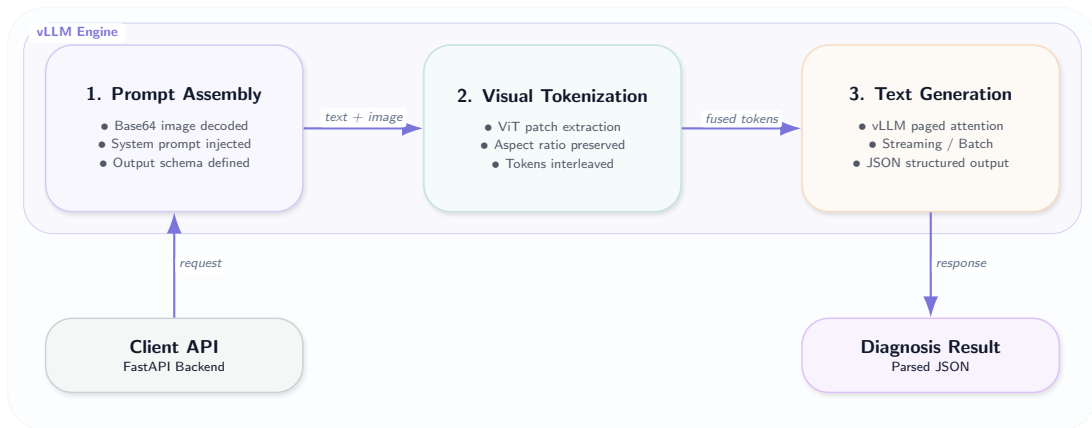
PEFT / LoRA tuning of Qwen2.5-VL-3B with visual tokens, distributed via Accelerate, logging to W&B, culminating in adapter merging.



Architecture — Training leverages parameter-efficient fine-tuning (PEFT) with LoRA to adapt the vision and language projection layers without modifying the massive base model. Bfloat16 precision prevents underflow issues common in visual token scaling.

VLM Inference Pipeline Flow

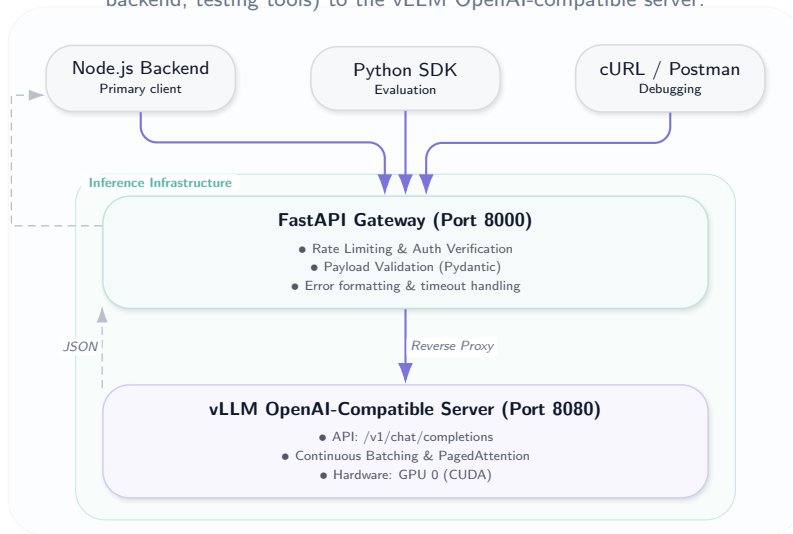
Local generation with vLLM server: Image parsing, dynamic visual tokenization, LLM text generation, and JSON constraint extraction.



Execution — The vLLM engine hosts the merged model. Instead of stretching the image, Qwen's visual architecture dynamically tokens the image patches based on resolution, feeding them alongside the text prompt. The system instructions enforce a strict JSON structure for the backend to ingest.

VLM API Gateway Flow

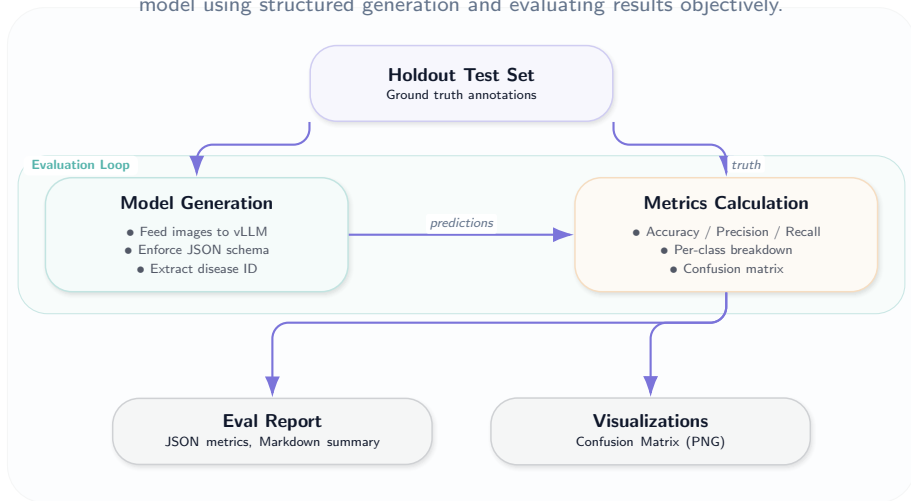
FastAPI reverse proxy routing traffic from multiple clients (Node.js backend, testing tools) to the vLLM OpenAI-compatible server.



Separation — The FastAPI gateway acts as a robust front door. The internal vLLM server is never exposed directly to clients; this prevents malformed requests from crashing the GPU worker and provides a layer for metrics and API key validation.

VLM Validation & Evaluation Flow

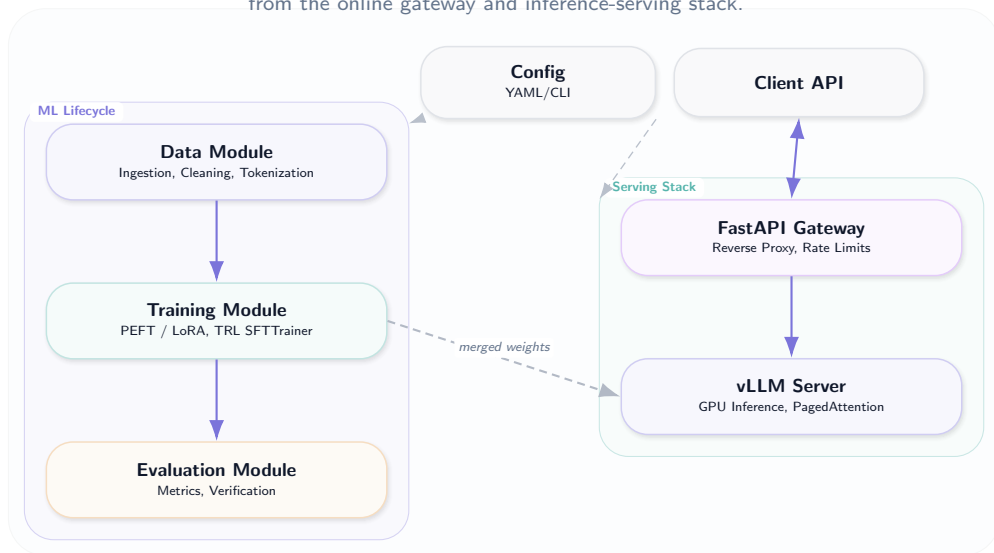
Fork-join evaluation pipeline: Running automated test sets against the model using structured generation and evaluating results objectively.



Process — Validation ensures the LoRA weights actually learned the agricultural domain by testing against a pristine holdout set. Because generation is constrained to JSON, predictions can be programmatically compared to ground truth without human grading.

VLM System Architecture Overview

The modular VLM architecture separates offline data and training workflows from the online gateway and inference-serving stack.



Architecture — The codebase uses a strict separation of concerns. Offline ML pipelines (data, train, eval) run independently from the online serving stack (gateway, vLLM). Configuration is injected globally via YAML.

API Routing & Middleware Flow

Express.js bootstrap and ordered middleware registration
with public vs protected route dispatch plus the layered request path.

Server Bootstrap

server.ts

1. Load .env via dotenv
2. Validate config (Zod schema)
3. Connect MongoDB Atlas
4. Register DI container (repos, services, controllers)
5. Create Express app
6. Listen on PORT

Middleware Pipeline

helmet

cors

rateLimit

body parsers

morgan

Route Dispatch

Route Matcher

/api/v1/*

no auth

authMiddleware

Public Routes

GET /health → liveness
POST /auth/* → register,
login, refresh, OTP
POST /diagnose → diagnosis

Protected Routes

JWT

/users → profile CRUD
/crops → crop management
/scans → scan records
/tasks, /notes, /advisories

Controller Layer — validate request body → invoke service method → shape JSON response

Processing Chain

Service
business rules

DTO

Repository
data access

ODM

MongoDB
document store

catch

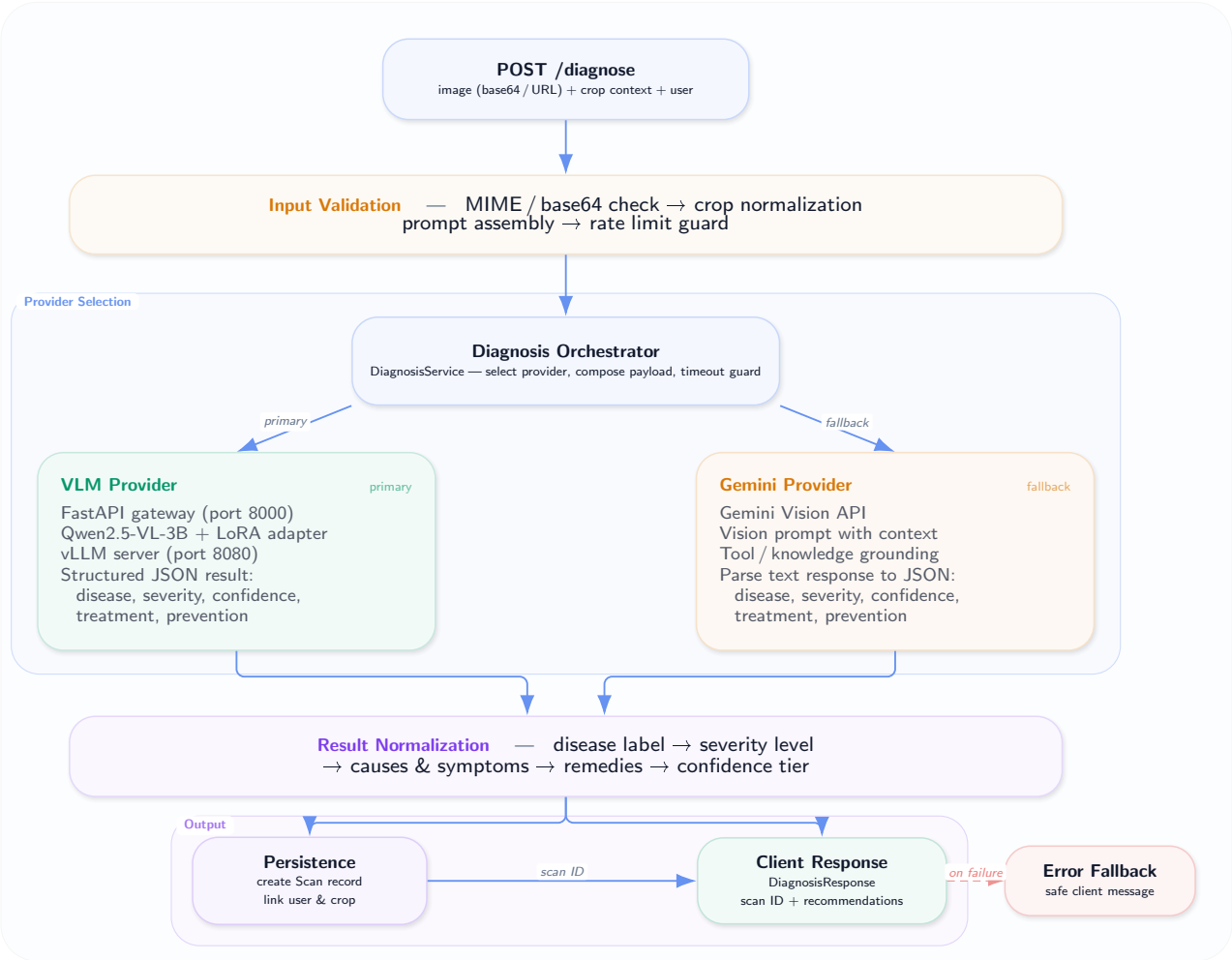
Error Handler
ApiError → JSON

JSON Response
structured envelope

Design rationale — Middleware executes in strict registration order. Public routes (/health, /diagnose, /auth/*) bypass JWT verification; protected routes inject req.user via authMiddleware. Controller → Service → Repository → MongoDB enforces separation of validation, business logic, and persistence.

Diagnosis Pipeline Flow

Image diagnosis orchestration across validation, dual inference providers (VLM primary / Gemini fallback), result normalization, persistence, and response delivery.



Critical path — The backend hides provider differences behind a single `DiagnosisService` interface so the mobile client receives one stable diagnosis contract regardless of which inference engine produced the result. VLM is the primary provider; Gemini is activated as a fallback when the VLM gateway is unreachable or returns an error.

Data Models & Persistence Flow

MongoDB schemas, generic repository abstraction, dependency-injected services, and the write/read data paths that keep domain state consistent.

MongoDB Schemas

User
profile, hash

Crop
plot, variety

Scan
image, result

Task
field work

Note
observation

Advisory
guidance

OTP
recovery

RefreshToken
session

Repository Layer — BaseRepository<T> provides typed CRUD helpers; concrete repos (UserRepo, CropRepo, ScanRepo, TaskRepo, NoteRepo, AdvisoryRepo) add domain-specific queries

DI Container

register repos → services → controllers
single construction point at bootstrap

Service Layer

AuthService
tokens, OTP

UserService
profile ops

CropService
plot CRUD

ScanService
diagnosis

Data Paths

Write Path

validate DTO → service rule
→ repo mutation → model save

Read Path

route params → repo query
→ ownership check → serialize

MongoDB Atlas

MONGO_URI — connection lifecycle, indexes, production datastore

Architecture — Controllers stay thin; services own business rules; repositories isolate persistence mechanics behind BaseRepository<T>. The DI container wires everything at bootstrap so each layer depends only on abstractions, not concrete implementations. JWT user context from req.user propagates through the service layer for ownership checks.

Authentication & User Management Flow

Registration, login, JWT token lifecycle, refresh rotation, OTP password recovery, and route authorization via auth middleware.

Identity Establishment

Registration

- Validate identity fields
- Hash password (bcrypt)
- Create User document
- Optional OTP email

credentials

Login

- Verify email + password
- Issue access token (JWT)
- Store refresh token
- Return user profile

JWT issued

Protected Request

- Extract Bearer token
- Decode & verify JWT
- Attach req.user
- Proceed to controller

token ops

forgot password

middleware result

Token & Recovery Lifecycle

Token Lifecycle

- Short-lived access token
- Refresh token rotation
- Logout single session
- Logout all sessions
- Invalidate on password change

token reset

Password Recovery

- Send OTP to email
- Verify OTP code
- Reset password
- Invalidate stale tokens
- Rate-limited attempts

re-auth

Auth Outcome

- 200 authorized controller call
- 401 missing/expired token
- 403 invalid ownership
- 429 rate limit exceeded
- Global error handler

Middleware Decision

authMiddleware

Decode JWT → verify expiry → lookup user → attach req.user → next()

valid

Authorized
controller proceeds

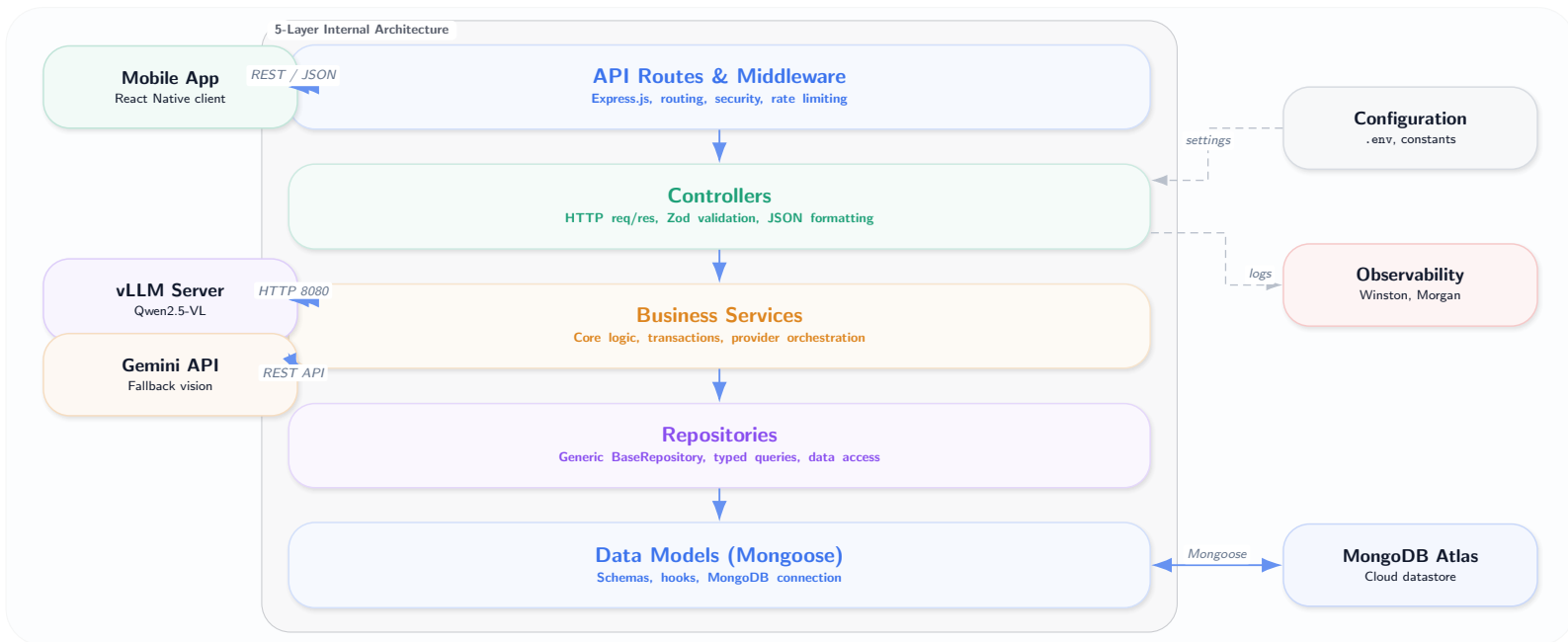
invalid

Denied
401 / 403 JSON error

Security — Session state is explicit: refresh tokens can be individually revoked without invalidating the stateless access-token verification path. The OTP recovery flow is rate-limited and time-bounded to prevent brute-force attacks. All sensitive operations log an audit trail.

System Architecture Overview

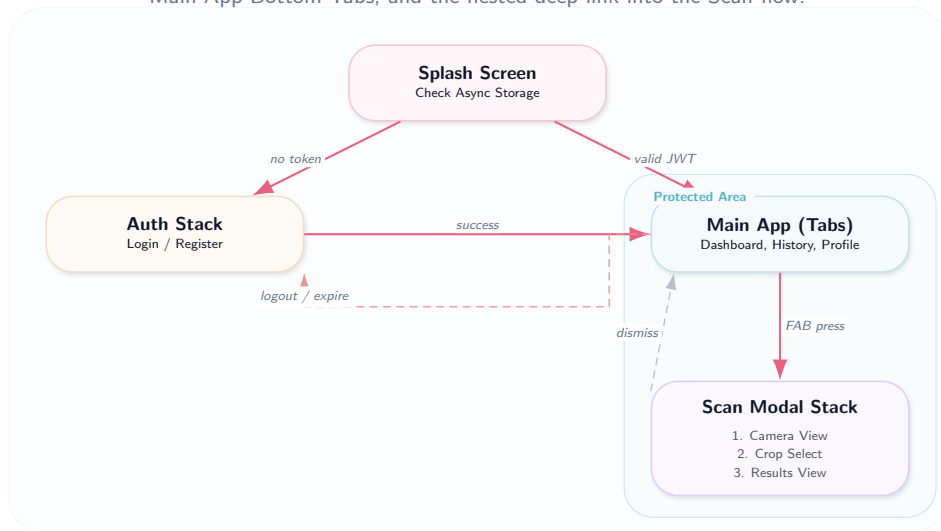
The internal request stack from routes through data access, together with the external services and operational wiring around the backend API.



Separation of Concerns — The 5-layer architecture ensures HTTP logic (Controllers) is isolated from business logic (Services), which is isolated from database mechanics (Repositories/Models). Cross-cutting concerns like configuration and logging are injected throughout.

User Navigation & Routing Flow

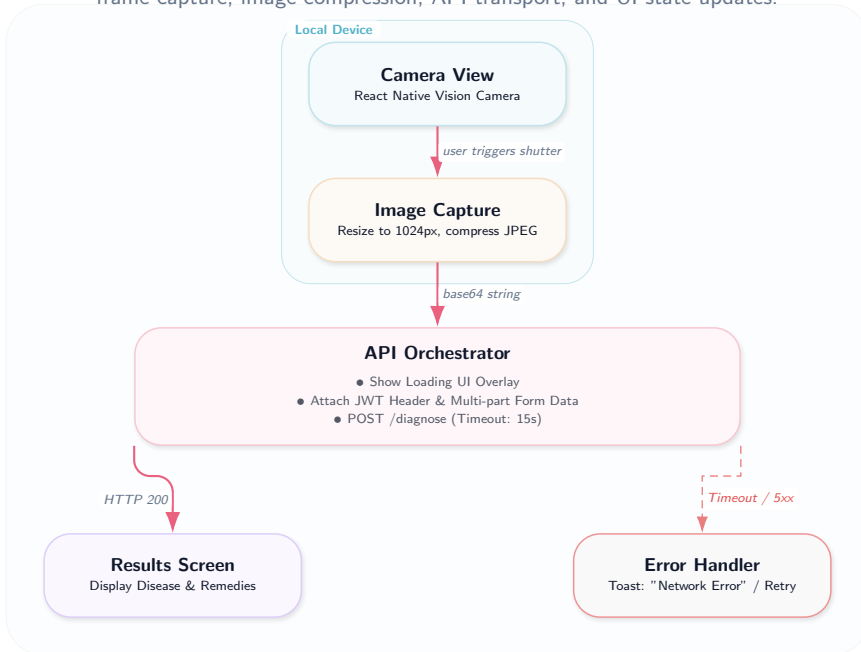
React Navigation state machine: Switch from Splash/Auth stack to the Main App Bottom Tabs, and the nested deep link into the Scan flow.



Navigation Hierarchy — The Root Navigator uses conditional rendering to swap between the Auth Stack and the Main App Stack based on the JWT state, ensuring unauthenticated users cannot deep-link into protected tabs. The Scan Stack is presented modally over the bottom tabs to preserve context.

Mobile App Scan Pipeline Flow

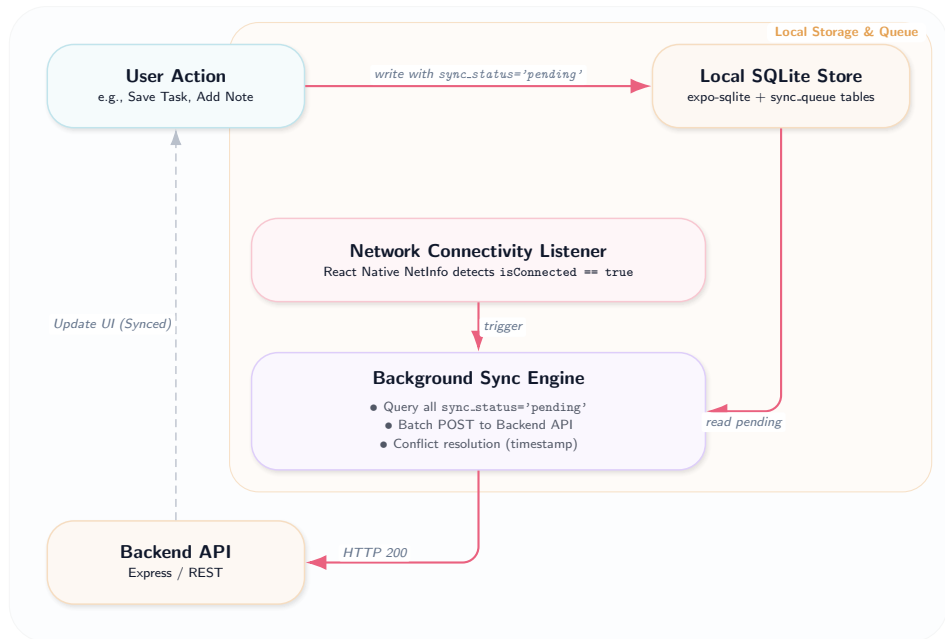
The user journey from opening the camera to receiving a diagnosis, handling frame capture, image compression, API transport, and UI state updates.



UX Considerations — Image compression happens natively on the thread to prevent UI freezing. During the API call, a custom skeleton loader or Lottie animation is shown to keep the user engaged while waiting for the VLM backend inference.

Mobile Offline Sync Data Flow

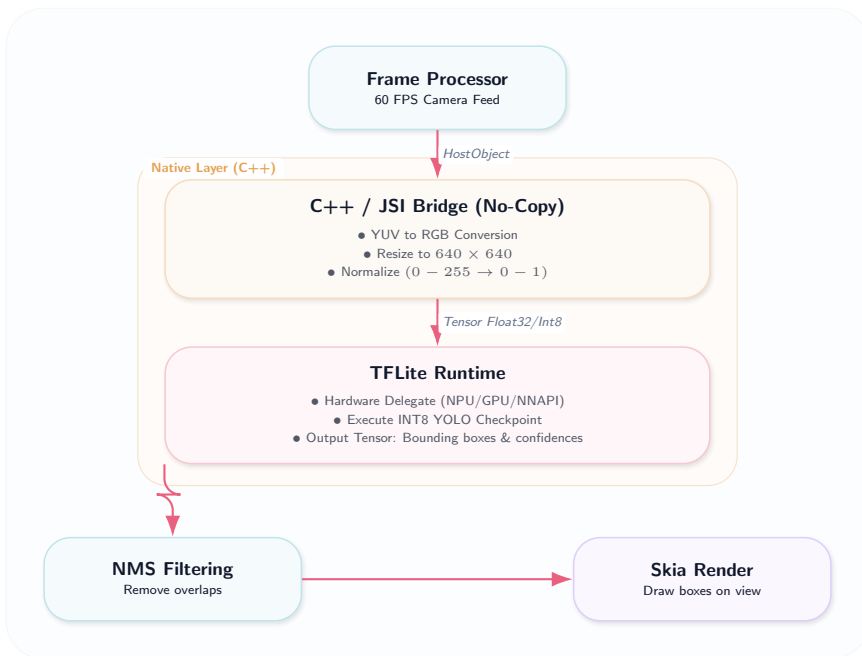
Handling user actions without internet connectivity using the local SQLite store and queued sync operations, then flushing them when network connectivity returns.



Offline-First — The UI always reads from and writes to the local SQLite-backed data layer, ensuring zero latency and full offline support. The Sync Engine runs as a background job that pushes local mutations to the cloud and pulls fresh data down when the network is available.

On-Device ML Inference Flow (YOLO Edge)

Live camera frame extraction, no-copy preprocessing via JSI, and hardware-accelerated tensor execution for real-time bounding-box overlays.



Performance — To achieve real-time (30+ FPS) AR overlays, frames are never serialized to base64 or passed over the React Native bridge. Instead, C++ JSI bindings manage the memory pointers directly from the camera buffer to the TFLite runtime, preventing garbage collection stutters.

Mobile System Architecture Overview

Layered React Native architecture spanning presentation, Zustand-driven state, native bridges, and the app's external API plus local persistence boundaries.



Architecture — The mobile application strictly separates presentation from state and side effects. React Query handles API caching and loading states, while Zustand manages synchronous UI state. Performance-critical operations (storage, camera, ML) bypass the JS bridge using JSI.